

The Production-Grade Vibe Coding Checklist

For a FastAPI + Postgres + LangChain/LangGraph + Qdrant + Docker AI SaaS

How to use this doc: Build one module at a time. For each module, paste the relevant section to your AI coding agent and say *"implement this module following these exact requirements, and explain any deviation."* Then come back here and manually verify the checklist items before moving to the next module. Don't let the AI build three modules at once — you can't review what you can't see.

WHY THIS MATTERS (read once)

AI coding agents are trained to produce code that *looks* correct and *runs* without errors. They are not automatically trained to produce code that is *secure*, *scalable*, or *cost-safe*. The failure mode of vibe coding isn't "the app crashes" — it's "the app works perfectly in your demo and then leaks user data / racks up a \$4,000 OpenAI bill / falls over at 50 concurrent users" three weeks later. Every item below exists because it's a real, common way that happens.

TIER 1 — FOUNDATIONS (do these before writing any feature code)

1. Environment & Secrets Management

Check	Correct config	Why / what breaks without it
Secrets never hardcoded	Use <code>.env</code> locally (via <code>pydantic-settings</code> / <code>python-dotenv</code>), and a real secrets manager in prod (Docker/K8s secrets, AWS Secrets Manager, Doppler, Infisical)	If an AI writes <code>API_KEY = "sk-abc123"</code> directly in code and you push to GitHub, it's scraped by bots within minutes. Real incidents happen this way constantly.
<code>.env</code> is in <code>.gitignore</code> from commit #1	<code>.gitignore</code> includes <code>.env</code> , <code>.env.*</code> , <code>*.pem</code> , <code>*.key</code>	AI agents frequently forget this because they're focused on the feature, not the repo hygiene.
Separate configs per environment	<code>Settings</code> class with <code>ENV=dev/staging/prod</code> , different DB URLs, different LLM rate limits per env	Prevents your dev experiments from hitting production data or your prod API keys.
No secrets in Docker image layers	Use <code>--build-arg</code> only for non-secrets; inject secrets at runtime via env vars or mounted secrets, never <code>COPY .env</code> into the image	Docker images are often pushed to registries (GHCR) — anyone who pulls the image can extract secrets from any layer, even "deleted" ones.
<code>pydantic-settings</code> validates required env vars at startup	App should fail fast (crash on boot) if a required secret is missing, not fail silently later	Silent failure = a request fails in production at 2am instead of your deploy failing in CI where you'd notice.

Ask the AI: "Set up a `pydantic-settings` based config system with `dev/staging/prod` environments, validate all required secrets at startup, and make sure nothing sensitive can end up in the Docker image or git history."

2. Database Design & ORM (Postgres)

Check	Correct config	Why / what breaks without it
Use an ORM with migrations	SQLAlchemy 2.0 (async) + Alembic for migrations	Without migrations, schema changes are manual SQL run by hand — easy to desync dev/staging/prod. AI agents frequently "forget" migrations and just tell you to <code>DROP TABLE</code> and recreate, which is catastrophic in prod.
Primary keys	Prefer <code>UUID</code> (v4 or v7) over auto-increment <code>int</code> for anything user-facing	Auto-increment IDs leak business info (guess how many users you have) and are enumerable (<code>/users/1</code> , <code>/users/2</code> ...). If you do use int, always set it as <code>SERIAL</code> / <code>IDENTITY</code> , never a manually managed counter.
Nullable fields explicit	Every column should have <code>nullable=False</code> unless it's genuinely optional	AI agents default to <code>nullable=True</code> out of laziness. This lets bad data (<code>email = NULL</code>) into your DB silently, causing crashes downstream when code assumes email exists.
Constraints at the DB level, not just app level	<code>CHECK</code> constraints (e.g., <code>price >= 0</code>), <code>UNIQUE</code> , <code>FOREIGN KEY</code> with <code>ON DELETE</code> behavior explicitly chosen (<code>CASCADE</code> / <code>RESTRICT</code> / <code>SET NULL</code>)	App-level validation can be bypassed (direct DB access, race conditions, bugs). DB constraints are the last line of defense and are non-negotiable for financial/quantity fields.
Default values set explicitly	e.g., <code>created_at=func.now()</code> , <code>is_active=True</code> , <code>credits=0</code>	Prevents <code>NULL</code> creeping into places your code doesn't expect.
Indexes on foreign keys and frequently filtered columns	<code>CREATE INDEX</code> on any column used in <code>WHERE</code> , <code>JOIN</code> , or <code>ORDER BY</code> at scale	Without this, queries that are fast with 100 rows become painfully slow with 100,000 rows. AI agents rarely add indexes unless told to.
Connection pooling configured	SQLAlchemy <code>pool_size=5-20</code> , <code>max_overflow=10</code> , <code>pool_pre_ping=True</code> , <code>pool_recycle=1800</code>	Without pooling limits, under load you either exhaust Postgres's max connections (default 100) and crash the DB, or leak stale/dead connections.
Transactions used for multi-	Wrap related writes in a single <code>async with session.begin():</code>	Without this, a crash mid-operation leaves your DB in a half-

Check	Correct config	Why / what breaks without it
step writes		updated, inconsistent state (e.g., payment charged but credits not added).
Soft deletes vs hard deletes decided intentionally	Use <code>deleted_at TIMESTAMP NULL</code> for user data you may need to recover/audit; hard delete only for genuinely disposable data	AI defaults to hard <code>DELETE</code> . For a SaaS, accidental data loss (bug, user error) with no recovery path is a common way to lose a customer's trust permanently.
Never trust AI-generated raw SQL blindly	If the AI writes raw SQL (not ORM), check for string concatenation of user input	This is the #1 SQL injection source. Always demand parameterized queries.

Ask the AI: "Design the database schema using SQLAlchemy 2.0 async with Alembic migrations. Use UUID primary keys, explicit nullable=False/True on every column, CHECK constraints for any numeric field that shouldn't go negative, and explicit ON DELETE behavior for every foreign key. Show me the constraint list explicitly before writing code."

3. Input Validation

Check	Correct config	Why
Every endpoint uses Pydantic models for input	Never accept raw <code>dict</code> or untyped <code>**kwargs</code> from request bodies	Pydantic gives you free type coercion, validation, and OpenAPI docs. Skipping it means malformed data reaches your business logic or DB.
Field-level constraints	<code>Field(min_length=1, max_length=255)</code> , <code>Field(gt=0)</code> , <code>EmailStr</code> , regex patterns for things like usernames	Prevents empty strings, absurdly long inputs (DoS via huge payloads), negative quantities, etc.
Reject unknown fields (optional but recommended for strict APIs)	<code>model_config = ConfigDict(extra="forbid")</code>	Prevents silent typos in client requests and mass-assignment style bugs.
File upload validation	Check MIME type, file size limit, and re-validate extension (don't trust filename)	AI often forgets size limits — a single unrestricted upload endpoint can be used to fill your disk or crash your server.

TIER 2 — SECURITY BASICS

4. Authentication (JWT)

Check	Correct config	Why / what breaks without it
Algorithm	Use RS256 or ES256 (asymmetric) for anything beyond a toy project, not HS256	HS256 uses one shared secret to sign <i>and</i> verify — if any service that verifies tokens is compromised, attackers can forge tokens. RS256/ES256 let you keep the private (signing) key only on your auth server, and distribute the public key freely for verification. If you must use HS256, the secret must be ≥32 random bytes, never a guessable string.
Access token expiry	15 minutes	AI agents commonly default to no expiry or 7+ days "for convenience." A long-lived access token that leaks (XSS, logs, browser history) is a long-lived vulnerability.
Refresh token expiry	7–30 days , stored securely (httpOnly cookie, not localStorage)	Refresh tokens let you keep access tokens short without forcing constant re-login. Storing them in <code>localStorage</code> exposes them to XSS attacks — always use httpOnly, Secure, SameSite=Strict cookies.
Refresh token rotation	Issue a new refresh token every time one is used, invalidate the old one	Without rotation, a stolen refresh token is valid indefinitely. With rotation, reuse of an old token is a signal of theft and can trigger auto-revocation of the whole session.
Token revocation / logout	Maintain a denylist (Redis, short TTL matching token expiry) or use short-lived tokens + refresh rotation as the primary defense	JWTs are stateless by design — "logout" doesn't actually invalidate them unless you build this. Many AI-built demos have a fake logout that just deletes the token client-side while the token is still valid server-side.
Password hashing	bcrypt (cost factor 12) or argon2id — never MD5, SHA-1, SHA-256 alone, or plaintext	Fast hash functions (SHA-256) are designed for speed, which is exactly wrong for passwords — attackers can brute-force billions/sec. bcrypt/argon2 are deliberately slow and salted.
<code>sub</code> , <code>exp</code> , <code>iat</code> , <code>iss</code> , <code>aud</code> claims all set	Standard JWT claims, validated on every request	Missing <code>exp</code> = token never expires. Missing <code>aud</code> / <code>iss</code> validation = a token meant for a different service could be replayed against yours.
Sensitive data never in the JWT payload	Only put user ID + role/claims needed for authorization — never email, password hash, PII	JWT payloads are base64-encoded, not encrypted — anyone can decode and read them.

Ask the AI: "Implement JWT auth using RS256, 15-minute access tokens, 7-day rotating refresh tokens stored in httpOnly cookies, bcrypt (cost=12) password hashing, and a Redis-based revocation list for logout. Show me exactly what claims go in the token."

5. Authorization (RBAC / permissions)

Check	Correct config	Why
Role/permission check on every protected route	Explicit dependency (FastAPI <code>Depends</code>) checking role, not just "is logged in"	AI often implements authentication (who are you) but forgets authorization (what are you allowed to do) — leading to any logged-in user being able to hit admin endpoints.
Object-level authorization	Check the resource being accessed actually belongs to the requesting user (e.g., <code>document.owner_id == current_user.id</code>)	This is the single most common real-world SaaS vulnerability (IDOR — Insecure Direct Object Reference): User A can view/edit User B's data just by changing an ID in the URL. AI-generated CRUD endpoints almost never check ownership unless explicitly told to.
Principle of least privilege for DB users too	The app's DB user shouldn't have <code>DROP</code> / <code>CREATE</code> privileges in production	Limits blast radius if the app itself is compromised via a bug.

6. Rate Limiting

Check	Correct config	Why
Per-user and per-IP limits	e.g., <code>slowapi</code> or Redis-based sliding window: 60–100 req/min general API, much stricter on auth endpoints (5 login attempts / 15 min)	Without this, your login endpoint is a free brute-force target, and your LLM endpoints are a free way for someone to run your OpenAI/Anthropic bill to zero.
Separate, stricter limits on LLM/AI endpoints	e.g., 10–20 requests/min per user on chat/completion endpoints, plus a daily token/cost cap per user	LLM calls cost real money per request — this is the #1 way vibe-coded AI SaaS apps get "wallet-drained" by a bad actor or bug (infinite retry loop).
429 responses include <code>Retry-After</code> header	Standard HTTP practice	Lets well-behaved clients back off correctly instead of hammering you.
Rate limit state survives restarts / works across multiple instances	Backed by Redis, not in-memory dict	In-memory rate limiting resets on every deploy and doesn't work at all once you scale to 2+ app instances behind a load balancer — each instance would track limits separately, silently allowing 2x the traffic.

7. CORS & Security Headers

Check	Correct config	Why
CORS	Explicit <code>allow_origins=["https://yourdomain.com"]</code> — never <code>["*"]</code> in production, especially with <code>allow_credentials=True</code>	<code>allow_origins=["*"]</code> + credentials is a common AI default and lets <i>any</i> website make authenticated requests to your API on behalf of a logged-in user.
Security headers	<code>Strict-Transport-Security</code> , <code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: DENY</code> , <code>Content-Security-Policy</code> — use <code>fastapi</code> + a middleware like <code>secure</code> package	Missing headers open you up to clickjacking, MIME-sniffing attacks, and downgrade attacks.
HTTPS enforced	Redirect all HTTP → HTTPS at the reverse proxy (nginx/Caddy/Traefik) or load balancer level	Without this, tokens and passwords can be intercepted over plain HTTP (MITM).

TIER 3 — RELIABILITY

8. Error Handling

Check	Correct config	Why
Global exception handler	Catch-all handler that returns a generic error to the client but logs full details server-side	AI often either lets raw Python tracebacks leak to the client (exposes file paths, library versions, sometimes DB queries — a gift to attackers) or swallows errors silently with a bare <code>except: pass</code> . Neither is acceptable.
Custom exception classes for business logic	e.g., <code>InsufficientCreditsError</code> , <code>ResourceNotFoundError</code> mapped to proper HTTP status codes (404, 402, 409, etc.)	Using <code>500 Internal Server Error</code> for everything (including "user not found") makes debugging and client-side handling much harder.
No bare <code>except Exception: pass</code>	Always log the exception, even if you re-raise or handle gracefully	Silent failures are the hardest bugs to find — the AI will happily write these to "make the error go away" without fixing the root cause.
Retries with backoff for external calls (LLM APIs, Qdrant, third-party APIs)	Exponential backoff with jitter, max 3-5 retries, only on retryable errors (timeouts, 5xx, 429) — libraries: <code>tenacity</code>	Without this, a transient network blip becomes a user-facing failure. Without limits, a retry loop on a truly broken dependency can hammer that dependency (or your own rate limits) into the ground.

9. Logging

Check	Correct config	Why
Structured logging	JSON logs (<code>struct log</code> or Python's <code>logging</code> with a JSON formatter), not <code>print()</code>	AI defaults to <code>print()</code> for debugging and leaves it in. Structured logs are searchable/filterable in production (e.g., in Grafana Loki, Datadog); <code>print()</code> output is not.
No sensitive data in logs	Never log passwords, full JWTs, API keys, credit card numbers, or full PII	This is a common compliance violation (GDPR) and security risk — logs are often kept for months and read by more people than the DB is.
Correlation / request IDs	Generate a unique <code>request_id</code> per request, include it in every log line for that request (via middleware + <code>contextvars</code>)	Without this, tracing a single user's request through multiple log lines/services is nearly impossible once you have real traffic.
Log levels used correctly	DEBUG for dev detail, INFO for normal flow, WARNING for recoverable issues, ERROR for failures, CRITICAL for outages	AI often logs everything at INFO or DEBUG, drowning real problems in noise.

10. Health Checks & Graceful Shutdown

Check	Correct config	Why
<code>/health</code> (liveness) and <code>/ready</code> (readiness) endpoints	<code>/health</code> = "is the process alive," <code>/ready</code> = "can it actually serve traffic" (checks DB, Qdrant, Redis connectivity)	Load balancers and orchestrators (Docker, Kubernetes) use these to decide whether to route traffic to an instance or restart it. Without a real readiness check, traffic gets routed to instances that are up but can't reach the DB.
Graceful shutdown	Handle <code>SIGTERM</code> : stop accepting new requests, finish in-flight requests, close DB/Redis connections cleanly	Without this, every deploy/restart abruptly kills in-flight requests — users see random failures on every deploy.

TIER 4 — OBSERVABILITY & TESTING

11. Monitoring & Metrics

Check	Correct config	Why
Metrics exposed	Prometheus-format <code>/metrics</code> endpoint (via <code>prometheus-fastapi-instrumentator</code>): request count, latency histograms, error rates	Without metrics, you find out about a problem when a user complains, not when it starts.
LLM-specific metrics tracked	Tokens used per request, cost per request, latency per model call, error/timeout rate per provider	This is the AI-app-specific blind spot — teams often have zero visibility into LLM spend until the bill arrives. Track this from day one.
Alerting configured	Alerts on error rate spikes, latency spikes, DB connection pool exhaustion, unusual LLM spend	Metrics without alerts just sit in a dashboard nobody watches at 3am.
Distributed tracing (once you have multiple services)	OpenTelemetry, exported to something like Jaeger/Honeycomb/Datadog	Lets you see the full path of a request across FastAPI → LangGraph → Qdrant → Postgres, critical once agentic workflows have multiple steps.

12. Testing

Check	Correct config	Why
Unit tests for business logic	<code>pytest</code> , aim for coverage on anything with branching logic, not just happy paths	AI-generated tests often only test the happy path and are written to pass, not to catch bugs — review them critically, don't just trust "tests pass."
Integration tests for API endpoints	<code>httpx.AsyncClient</code> + a test DB (via <code>testcontainers</code> or a dedicated test Postgres)	Confirms the full request→DB→response path works, not just isolated functions.
Auth/authorization tests explicitly	Test that User A cannot access User B's resources, that expired tokens are rejected, that unauthenticated requests are blocked	This is exactly the class of bug (IDOR, missing auth) that's easiest for AI to introduce and easiest to miss without a dedicated test.
CI runs tests on every PR	GitHub Actions workflow blocks merge on failing tests	Without this, "tests exist" doesn't mean "tests are actually enforced."

TIER 5 — AI/LLM-SPECIFIC (LangChain, LangGraph, Qdrant)

13. LLM Cost & Reliability Control

Check	Correct config	Why
Token limits set explicitly	<code>max_tokens</code> set on every LLM call, not left to provider default	Without this, a single malformed prompt or loop can generate extremely long (expensive) outputs.
Per-user quota / daily cost cap	Track tokens or \$ spent per user per day, hard-stop when exceeded	This is your actual safety net against a runaway bill — rate limiting alone isn't enough if each request is expensive.
Timeouts on every LLM call	e.g., 30–60s timeout with a fallback response	Without a timeout, one hung LLM call can hold a worker/connection indefinitely, degrading service for everyone.
Streaming used for chat UX	Use provider streaming APIs so users see tokens as they generate	Not a hard requirement, but its absence is a common "why does this feel like a toy" complaint — and long non-streamed calls are more likely to hit gateway timeouts.
Fallback / circuit breaker for LLM provider outages	If primary provider fails repeatedly, fail fast with a clear error (or fall back to a secondary model) rather than retrying forever	Prevents cascading failures when an upstream provider (OpenAI/Anthropic) has an outage.
Prompt injection awareness	Never directly concatenate untrusted user input into a system prompt that controls sensitive actions (tool calls, DB writes); validate/sandbox what tools an agent can call	LangGraph agents that can call tools (especially DB-writing or file-system tools) are a real attack surface — a user could craft input that manipulates the agent into calling tools it shouldn't.
LangGraph agent loops have a max iteration/step limit	e.g., <code>recursion_limit</code> set explicitly	Without this, a buggy agent graph can loop indefinitely, burning tokens and money until it hits an external timeout.

14. Qdrant / Vector DB

Check	Correct config	Why
Collection schema defined explicitly	Explicit vector size (matches your embedding model's dimension, e.g., 1536 for <code>(text-embedding-3-small)</code>), distance metric (usually <code>(Cosine)</code>)	Mismatched vector dimensions cause hard failures; wrong distance metric silently gives poor/irrelevant search results without erroring.
Embedding model version tracked/pinned	Store which embedding model generated each vector (e.g., in payload metadata)	If you ever change embedding models, old and new vectors are not comparable — mixing them silently degrades search quality with no error thrown.
Payload includes access-control metadata	e.g., <code>(user_id)</code> / <code>(org_id)</code> in the payload, filtered on every query	Without this, semantic search can return one user's private documents to another user — a serious data leak specific to RAG apps.
Indexing/quantization considered at scale	HNSW params tuned, or scalar/binary quantization for large collections	Default settings are fine to start; revisit once you have >1M vectors or latency issues.

TIER 6 — INFRA & DEVOPS

15. Docker

Check	Correct config	Why
Multi-stage builds	Separate build stage (compiles deps) from a slim runtime stage (<code>python:3.12-slim</code>)	Keeps final image small and avoids shipping build tools/compilers into production, reducing attack surface and image size significantly.
Non-root user in container	<code>USER appuser</code> set explicitly, not running as root	Running as root in a container means a container escape gives root on the host. AI defaults to root because it's simpler and "just works."
<code>.dockerignore</code> present	Excludes <code>.env</code> , <code>.git</code> , <code>__pycache__</code> , <code>tests/</code> , <code>node_modules</code>	Without it, secrets and bloat can leak into the image, and builds are slower.
Pinned base image versions	<code>python:3.12.4-slim</code> not <code>python:latest</code>	<code>latest</code> changes underneath you — a rebuild six months from now can silently break or introduce vulnerabilities/behavior changes.
Health check defined in Dockerfile/compose	<code>HEALTHCHECK</code> instruction pointing at your <code>/health</code> endpoint	Lets Docker/orchestrators detect and restart unhealthy containers automatically.

16. CI/CD (GitHub Actions + GHCR)

Check	Correct config	Why
Pipeline stages	lint → type-check (<code>mypy</code>)(<code>ruff</code>) → test → build → security scan → push to GHCR → deploy	A pipeline that only does "build and push" skips the checks that actually catch AI-introduced bugs before they reach prod.
Secrets via GitHub Encrypted Secrets	Never in the workflow YAML directly	Same principle as app secrets — workflow files are code, and code gets committed and viewed by others.
Image scanning before push	<code>trivy</code> or <code>docker scout</code> scan for known CVEs in dependencies/base image	Catches known-vulnerable packages before they ship — a very common gap in AI-scaffolded CI configs.
Immutable, traceable image tags	Tag images with git SHA (not just <code>latest</code>) — e.g., <code>ghcr.io/you/app:sha-abc1234</code>	<code>latest</code> -only tagging makes rollback guesswork; you can't be sure what code is actually running.
Branch protection + required checks	Main branch requires passing CI + at least one review before merge	Prevents an AI agent (or you, moving fast) from pushing straight to production-deploying main.

TIER 7 — ADVANCED PRODUCTION CONCERNS

17. Caching

Check	Correct config	Why
Redis for hot-path caching	Cache expensive DB queries or LLM responses (for identical/near-identical prompts) with sensible TTLs	Reduces DB load and, importantly for AI apps, can meaningfully cut LLM costs for repeated queries.
Cache invalidation strategy explicit	Know exactly when a cache entry is invalidated (on write, TTL expiry, etc.)	"Cache invalidation is one of the two hard problems in computer science" for a reason — stale cache bugs are notoriously confusing to debug.

18. Async / Background Tasks

Check	Correct config	Why
Long-running work (LLM chains, embeddings, report generation) offloaded	Use a task queue (Celery, or <code>arq</code> / <code>RQ</code> with Redis) instead of doing it inline in the request	Blocking a web request for 30+ seconds on an LLM chain ties up server resources and risks gateway timeouts.
Idempotency for retried tasks	Tasks check "has this already been done" before re-doing side effects (e.g., don't double-charge, don't send duplicate emails)	Task queues retry on failure by default — without idempotency, a transient failure + retry can cause duplicate actions.

19. Scalability

Check	Correct config	Why
App is stateless	No in-memory session/user state that would break with multiple instances — session state goes in Redis/DB	Required before you can run more than one instance behind a load balancer.
Database read replicas considered (once you have real load)	Route read-heavy queries to a replica	Not needed on day one, but design your data access layer so it's not painful to add later.

20. Backup & Disaster Recovery

Check	Correct config	Why
Automated Postgres backups	Daily automated backups with tested restore process, point-in-time recovery if your provider supports it	Untested backups are not backups — verify you can actually restore from one. AI-scaffolded projects almost never include this unless asked.
Backup retention policy	e.g., daily for 7 days, weekly for 4 weeks, monthly for 6 months	Balances recovery flexibility against storage cost.

21. API Versioning & Documentation

Check	Correct config	Why
Versioned API paths	<code>/api/v1/...</code> from day one	Lets you make breaking changes later without breaking existing clients.
OpenAPI docs kept accurate	FastAPI auto-generates from your Pydantic models/type hints — just make sure descriptions/examples are filled in, not left blank	Free documentation if you use proper types and docstrings; AI often skips descriptive detail, leaving auto-docs technically present but not actually useful.

22. Compliance & Data Privacy

Check	Correct config	Why
PII handling	Know exactly what personal data you store, encrypt sensitive fields at rest if needed, have a data deletion path (GDPR "right to be forgotten")	If you have EU users, this is a legal requirement, not optional. Even without legal exposure, it's a baseline trust requirement for a SaaS product.
Dependency vulnerability scanning ongoing	<code>(pip-audit)</code> or <code>(safety)</code> run in CI regularly, <code>(dependabot)</code> enabled on GitHub	Dependencies age and accumulate CVEs even if your code never changes — this needs to be continuous, not a one-time check.

HOW TO ACTUALLY USE THIS WITH AN AI AGENT

1. **One module per session/PR.** Don't ask for "build the whole backend" — ask for one Tier at a time. Smaller diffs are reviewable diffs.
2. **Ask the AI to state its choices, not just produce code.** e.g., *"Before writing code, tell me: what JWT algorithm and expiry will you use, and why?"* This surfaces bad defaults before they're baked into 500 lines of code.
3. **After every module, run through its table above like a checklist.** Literally ask the AI: *"Go through this checklist and tell me which items you've satisfied and which you haven't, honestly."* AI agents are often better at self-auditing against an explicit list than at remembering all the best practices unprompted.
4. **Treat AI-written tests with suspicion.** Ask it to specifically write tests for the *failure* and *abuse* cases (wrong user accessing data, expired token, negative quantity, empty input) — not just the happy path.
5. **Re-review security-critical modules (auth, authorization, payment) more than once,** ideally after the rest of the app is built and you can see how they're actually being called in practice.

Suggested build order

`(Tier 1 (env, DB, validation) → Tier 2 (auth, authz, rate limit, CORS) → Tier 3 (errors, logging, health) → Tier 6 (Docker, CI/CD – get deploys working early) → Tier 4 (tests, monitoring) → Tier 5 (LLM/Qdrant specifics) → Tier 7 (caching, async, scale, backup, compliance))`

Getting a boring, secure, observable skeleton deployed early — before your LangGraph agents get complex — makes every module after that dramatically easier to build and debug.